Comparative Analysis of Gradient Descent Methods in Predicting Stock Returns Using Financial Condition Indices

Zhengtai Zheng (PID:730511072)

STOR 512: Optimization for Machine Learning and Neural Networks

Spring 2024

Abstract

This study empirically investigates the efficiency of several gradient descent schemes applied to a binary classification problem with logistic regression, where the task involves using three subindices—risk, credit, and leverage of the National Financial Conditions Index (NFCI)—to predict the sign of weekly NASDAQ stock returns from January 12, 1999, to April 12, 2024. The findings indicate some level of directional predictability in stock market returns using historical data regarding financial condition, with the standard gradient descent method achieving a maximum accuracy of 70.4% on the testing data set beginning November 9, 2018. Several gradient descent schemes, including standard Gradient Descent (GD), Accelerated Gradient Descent (AGD), Stochastic Gradient Descent (SGD), and AdaGrad, were evaluated for their training speed and testing accuracy. The results reveal that while SGD and AdaGrad significantly increase training efficiency, they do so at the expense of some testing prediction accuracy in this particular dataset. In contrast, AGD, with a learning rate set as the inverse of the Lipschitz constant, moderately improves training efficiency without significantly compromising testing accuracy. Based on these outcomes, it is suggested that for such simple and straightforward classification tasks, AGD is considered as a balanced approach, which potentially offers both efficient training and reliable prediction accuracy.

Contents

1 Introduction 1
2 Data 3
3 Methodologies 5
3.1 Logistic Regression 5
3.2 Gradient Descent (GD) 5
3.3 Accelerated Gradient Descent (AGD) 6
3.4 Mini-batch Stochastic Gradient Descent (SGD) 7
3.5 Adaptive Subgradient (AdapGrad) 7
4 Results 7
5 Conclusion 10

Students' contribution

This project was completed by Zhengtai (Jimmy) Zheng (PID:730511072) independently. Zheng was solely responsible for data cleaning, coding, and writing the report. Without resorting to existent packages, he tried to manually develope functions for Gradient Descent (GD), Accelerated Gradient Descent (AGD), Stochastic Gradient Descent (SGD), and Adaptive Gradient (AdapGrad) to solve the binary classification problem via logistic regression. He also created plots to visualize training efficiency and test prediction accuracy. The Python code for this project is provided at the end of this document.

1 Introduction

One of the most classical methods for solving optimization problems (OP) is gradient descent (GD), originally proposed by Cauchy in 1847. Since its introduction, researchers have been developing algorithms to improve the the computational efficiency and accuracy. A significant enhancement is incorporating backtracking line search in the GD algorithm, which dynamically adjusts the learning rate to improve algorithm's convergence (Armijo, 1966). The accelerated gradient descent method is another variants of GD that offers faster convergence for convex functions by utilizing momentum-based updates (Nesterov,

1983). Furthermore, the mini-batch stochastic gradient descent (mini-batch SGD) leverages random sampling to allow updates based on subsets of data, thereby reducing the computational burden of gradient evaluation (Robbins & Monro, 1951). Adaptive subgradient (AdaGrad), developed by Duchi et al. (2011), is another common algorithm, adapting the learning rate to the parameters based on dual averaging and adaptive stepsize, provides a more sophisticated approach within the GD framework.

This paper empirically tests the efficiency of various GD and SGD algorithms in solving the binary classification problem of predicting the signs of weekly stock returns in the U.S. stock market. We consider the weekly closing price differences of the NASDAQ Composite index as the weekly stock returns of the U.S. stock market, an index that includes almost all stocks listed on the NASDAQ stock exchange. The aim is to use a straightforward dataset and a defined classification problem via logistic regression to empirically compare the efficiency of different GD and SGD algorithms.

Since the introduction of computers into economic forecasting, predicting stock returns has attracted significant interest. In financial economics theory, Kendall and Hill in 1953 were among the first to conclude there is no predictability in stock prices, and Fama in 1970 stated in the efficient market hypothesis that it should not be possible to achieve abnormal returns using historical information set. Given the difficulty of predicting the levels of returns, past research has also been utilizing machine learning methods to predict the binary sign of stock returns—essentially. Early support for using a binary response variable rather than actual levels was provided by Leung et al. (2000), and statistically significant predictability of direction in monthly stock returns has been reported by Nyberg (2011) and Christoffersen and Diebold (2006). Other studies also successfully predicted whether returns exceed a pre-determined threshold; for example, Linton and Whang (2007) using the estimated unconditional quantile of the return as this threshold.

To create a simple classification problem, this study explores whether publicly available macroeconomic and financial fundamentals can predict the direction of stock returns. It is motivated by the research of Bianchi et al. (2019), who found that worsening financial conditions and periods of illiquidity are linked to an increased predictability of anomaly stock returns. The study thus considered The Chicago Fed's National Financial Conditions Index (NFCI), developed by the Federal Reserve Bank of Chicago, as potential features in the logistic regression, which provides a comprehensive weekly update on U.S. financial conditions across money markets, debt and equity markets. Specifically, the three subindices of the NFCI—risk, credit, and leverage—offer a more granular analysis of NFCI movements and were used as features in the binary logistic regression.

The sections below are organized in the following way. Section 2 is a data description, which defines the outcome variables and features in the logistic regression equation, as

well as the training and testing splits for model estimation and evaluation. Section 3 introduces the binary logistic regression used to predict signs of stock market returns and outlines the algorithms for each of the GD schemes implemented in this study. Finally, Section 4 compares in training efficiency and testing prediction accuracy of the logistic regression solved by different GD schemes, and the conclusion is presented in Section 5.

2 Data

This study downloaded weekly returns of the NASDAQ Composite index from the Federal Reserve Economic Data (FRED) database, which span from January 12, 1999, to April 12, 2024, a total of 1,274 weeks. The first 1,000 weeks were chosen to be the training range, while the last 274 weeks (November 9, 2018 onwards) were the testing range. The research aimed to determine if historical data on financial conditions could predict the current trends in weekly stock returns. To assess these financial conditions, the Chicago Fed's National Financial Conditions Index (NFCI) was employed. This index updates three subindices weekly—risk, credit, and leverage—each normalized to have a mean of zero and a standard deviation of one, using data going back to 1973. The risk subindex reflects volatility and funding risks in the financial sector, the credit subindex measures credit availability, and the leverage subindex includes metrics like debt to equity ratios. A tighter or deteriorated financial condition is linked to higher risk, tighter credit, and lower leverage. Consequently, positive values of each subindex may correspond to tighter-than-average financial conditions while negative values indicate the opposite. Figure 1 and Figure 2 visualizes the weekly returns of NASDAQ Composite Index in the training and testing set respectively.

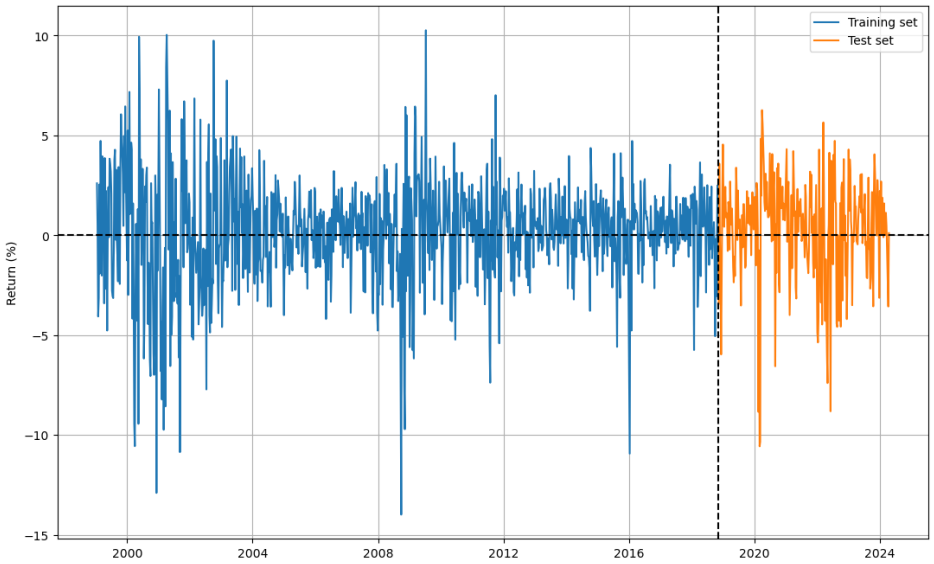


Figure 1: Weekly Returns of NASDAQ Composite Index (Training V.S. Testing)

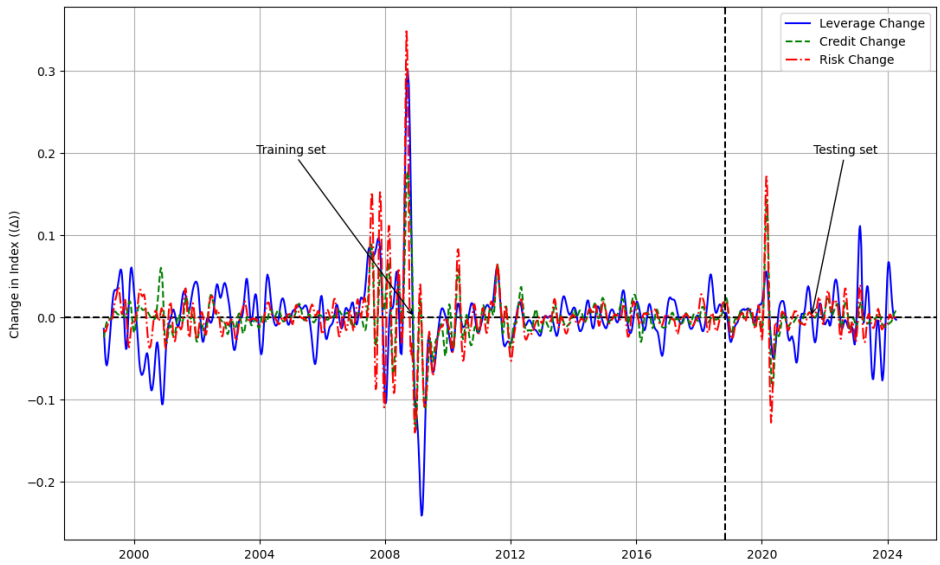


Figure 2: Weekly Returns of NASDAQ Composite Index (Training V.S. Testing)

3 Methodologies

3.1 Logistic Regression

Let y_t denote the binary outcome for the sign of NASDAQ returns at week t, taking values in $\{0, 1\}$, where 1 indicates a positive return and 0 indicates a negative return. The predictors $x_{i,t-1}$ for $i = 1, 2, 3$ are the lagged values of risk, credit, and leverage indices from the NFCI sub-indices respectively. The goal is to predict y_t based on past information of financial conditions, i.e. $\mathbf{x}_{t-1} = [x_{1,t-1}, x_{2,t-1}, x_{3,t-1}]^\top$.

The probability of $y_t = 1$ was modeled by using the logistic (sigmoid) function σ, which is given by:

$$\sigma\left(\mathbf{x}_{t-1}; \mathbf{w}\right) = \frac{1}{1 + e^{-\mathbf{w}^\top \mathbf{x}_{t-1}}}, \tag{1}$$

where $\mathbf{w} = [w_1, w_2, w_3]^\top$ is the vector of weights associated with each predictor (bias term is omitted for simplicity).

The loss function L, representing the (average) negative log-likelihood of observing the sequence of returns given the predictors, is defined as:

$$L(\mathbf{w}) = -\frac{1}{T} \sum_{t=1}^{T} \left[y_t \log\left(\sigma\left(\mathbf{x}_{t-1}; \mathbf{w}\right)\right) + (1 - y_t) \log\left(1 - \sigma\left(\mathbf{x}_{t-1}; \mathbf{w}\right)\right) \right], \tag{2}$$

where T is the training sample size. To minimize this loss function with respect to the weight vector $\mathbf{w}$, the gradient descent method was used. The gradient of L with respect to $\mathbf{w}$ is:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = \frac{1}{T} \sum_{t=1}^{T} \left[\sigma\left(\mathbf{x}_{t-1}; \mathbf{w}\right) - y_t \right] \mathbf{x}_{t-1} \tag{3}$$

The update rule for the weights in gradient descent at iteration n is given by:

$$\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} - \eta \nabla_{\mathbf{w}} L\left(\mathbf{w}^{(n)}\right), \tag{4}$$

where η is the learning rate. And five different GD algorithms to solve the minimization problem were implemented.

3.2 Gradient Descent (GD)

1. Initialize the weight vector $\mathbf{w} \in \mathbb{R}^d$ to $\mathbf{0}$, and set the learning rate $\eta = \left(\frac{1}{4}\lambda_{\max}\left(\mathbf{X}^\top \mathbf{X}\right)\right)^{-1}$, where $\lambda_{\max}$ denotes the maximum eigenvalue of $\mathbf{X}^\top \mathbf{X}$.

2. For each iteration n, compute the vector of probabilities $\mathbf{p}^{(n)} = \sigma\left(\mathbf{X}\mathbf{w}^{(n)}\right)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid function.

3. Update the loss history $L[n]$ with $L\left(\mathbf{w}^{(n)}\right) = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \log\left(p_i^{(n)}\right) + (1-y_i)\log\left(1-p_i^{(n)}\right)\right]$ and the error history error $[n]$ with $\frac{1}{m}\sum_{i=1}^{m}\left|y_i - 1\left\{p_i^{(n)} \geq 0.5\right\}\right|$

4. Compute the gradient $\nabla_{\mathbf{w}} L\left(\mathbf{w}^{(n)}\right) = \frac{1}{m}\mathbf{X}^{\top}\left(\mathbf{p}^{(n)} - \mathbf{y}\right)$, and update the weights $\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} - \eta\nabla_{\mathbf{w}} L\left(\mathbf{w}^{(n)}\right)$

5. If $|L[n] - L[n-1]| < 10^{-8}$, end.

3.3 Accelerated Gradient Descent (AGD)

1. Compute the vector of probabilities, and update the loss history $L[n]$, the error history, and the gradient as in Section 3.2.

2. Update the weights using the Nesterov accelerated step:

$$\mathbf{x}^{(n+1)} = \mathbf{y}^{(n)} - \eta\nabla_{\mathbf{w}} L(\mathbf{y}^{(n)}), \tag{5}$$

3. Update the sequence parameter:

$$\tau_{n+1} = \frac{1+\sqrt{1+4\tau_n^2}}{2}, \tag{6}$$

4. Update the auxiliary variable:

$$\mathbf{y}^{(n+1)} = \mathbf{x}^{(n+1)} + \left(\frac{\tau_n - 1}{\tau_{n+1}}\right)(\mathbf{x}^{(n+1)} - \mathbf{w}^{(n)}), \tag{7}$$

5. Set $\mathbf{w}^{(n+1)} = \mathbf{x}^{(n+1)}$.

6. If $|L[n] - L[n-1]| < 10^{-8}$, end

The backtracking line search of η was also implemented as an variant of AGD: if $n > 0$ and the current loss is not less than the previous loss, η is updated by multiplying with β. In the study, $\beta = 0.95$ was chosen.

3.4 Mini-batch Stochastic Gradient Descent (SGD)

1. The weight vector $\mathbf{w} \in \mathbb{R}^d$ is initialized to $\mathbf{0}$, and the learning rate η is set to $\left(\frac{1}{4}\lambda_{\max}\left(\mathbf{X}^\top\mathbf{X}\right)\right)^{-1}$. A batch size is chosen to segment the data into mini-batches.

2. For each iteration n, a mini-batch $\mathbf{X}_b$ and $\mathbf{y}_b$ are randomly sampled from the dataset with replacement. The vector of probabilities $\mathbf{p}^{(n)} = \sigma\left(\mathbf{X}_b\mathbf{w}^{(n)}\right)$ is computed, where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the logistic sigmoid function.

3. The loss history $L[n]$ is updated with

$$L\left(\mathbf{w}^{(n)}\right) = -\frac{1}{m}\sum_{i=1}^{m}\left[y_{b,i}\log\left(p_{b,i}^{(n)}\right) + (1-y_{b,i})\log\left(1-p_{b,i}^{(n)}\right)\right]$$

   and the error history error$[n]$ with $\frac{1}{m}\sum_{i=1}^{m}\left|y_{b,i} - 1\left\{p_{b,i}^{(n)} \geq 0.5\right\}\right|$.

4. The gradient $\nabla_{\mathbf{w}} L\left(\mathbf{w}^{(n)}\right) = \frac{1}{m}\mathbf{X}_b^\top\left(\mathbf{p}^{(n)} - \mathbf{y}_b\right)$ is computed, and the weights are updated using $\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} - \eta\nabla_{\mathbf{w}} L\left(\mathbf{w}^{(n)}\right)$.

5. If $|L[n] - L[n-1]| < 10^{-8}$, end.

3.5 Adaptive Subgradient (AdapGrad)

1. The weight vector $\mathbf{w} \in \mathbb{R}^d$ is initialized to $\mathbf{0}$. The learning rate η is set as $\left(\frac{1}{4}\lambda_{\max}\left(\mathbf{X}^\top\mathbf{X}\right)\right)^{-1}$. The vector $\mathbf{v} \in \mathbb{R}^d$ is initialized to $\mathbf{0}$ to keep track of the sum of squares of gradients.

2. For each iteration n, the following steps are executed:

   (a) Compute the vector of probabilities, and update the loss history $L[n]$, the error history, and the gradient as in Section 3.2.

   (b) The vector $\mathbf{v}$ is updated as $\mathbf{v}^{(n+1)} = \mathbf{v}^{(n)} + \left(\nabla_{\mathbf{w}} L\left(\mathbf{w}^{(n)}\right)\right)^2$.

   (c) The weights are updated according to the rule $\mathbf{w}^{(n+1)} = \mathbf{w}^{(n)} - \frac{\eta}{\sqrt{\mathbf{v}^{(n+1)}+\epsilon}} \circ \nabla_{\mathbf{w}} L\left(\mathbf{w}^{(n)}\right)$, where $\circ$ denotes the element-wise product.

3. If $|L[n] - L[n-1]| < 10^{-8}$, end.

4 Results

This section compares the training efficiency of various gradient descent methods in estimating the logistic regression model for predicting weekly NASDAQ returns. The standard GD using a fixed learning rate determined by the inverse of the Lipschitz constant

converged after 19,729 iterations (17.7 seconds). Although GD took relatively longer to converge compared to the others, it maintained a consistent and steady decrease in loss and training error reduction as shown in the Figure 3.

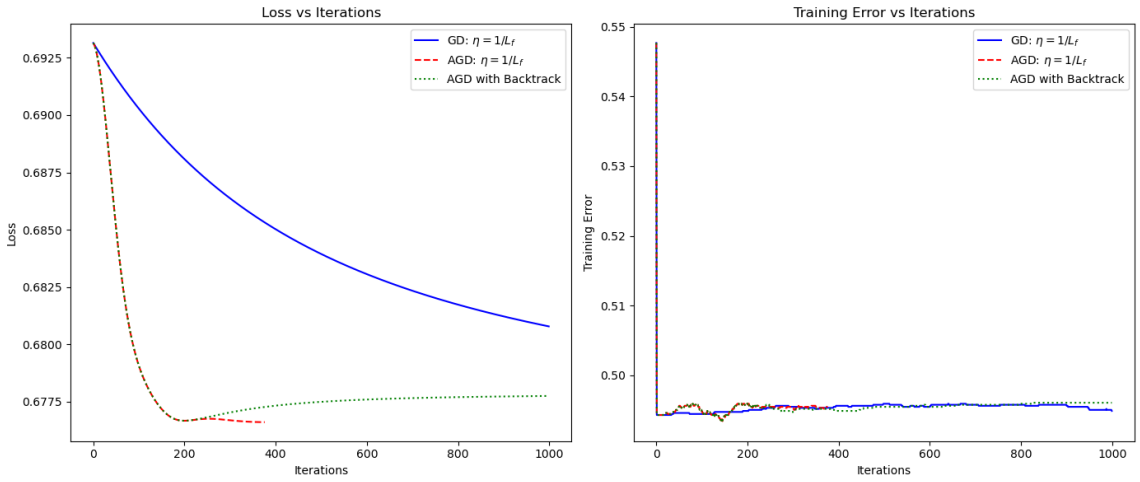


Figure 3: Training Efficiency of GD and AGD

Also shown in Figure 3, AGD, designed to speed up convergence in convex optimization by leveraging momentum, showed a significant improvement in convergence speed, taking only 474 iterations (0.43 seconds), but adding backtracking line search in learning rate to AGD resulted in longer convergence times than AGD alone (3,075 iterations, 2.76 seconds) but still significantly less than standard GD. It appears that additional complexity introduced by backtracking did not necessarily lead to improved training efficiency.

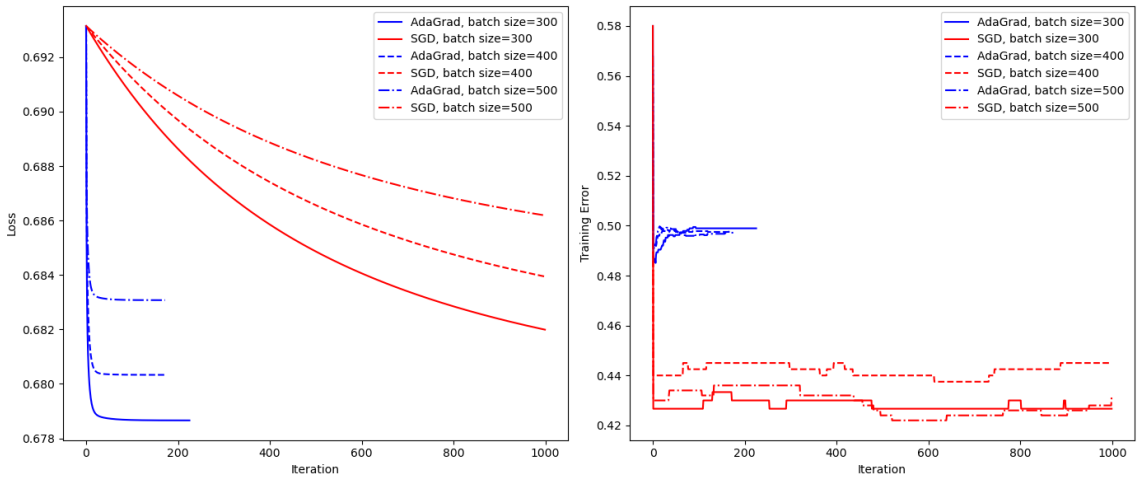


Figure 4: Training Efficiency of SGD and AdapGrad

On the other hand, according to Figure 4, SGD variants showed a diverse range of behaviors with different batch sizes, converging slowest (15,143 iterations, 0.61 seconds) with the largest batch size (50% of training sample) but with a lower loss and error compared to smaller batch sizes. SGD with batch size of 30% training sample converged in 8,458 iterations (0.29 seconds), whereas that with 40% converged in 8,854 iterations (0.33 seconds). Compared to GD, incorporating resampling scheme like SGD does significantly improves the training speeds from 17.7 seconds to 0.29 seconds. Finally, AdaGrad, known for its adaptive learning rates, showed significant fast convergence in all batch sizes tested, with the fastest being the largest batch size (40%), converging in just 175 iterations and 0.03 seconds.

The improved in training efficiency may come at a price of the prediction accuracy in testing data. Although slower than SGD and AdapGrad, it appears that both GD and AGD demonstrate relatively low testing error rates of 0.296 and 0.303, respectively. Despite AGD's significantly faster convergence in training, it does not result in a better testing error rate compared to the standard GD. AGD method shows a further increase in the testing error rate to 0.318 despite having a moderate training efficiency improvement over GD. SGD and AdaGrad with 30% batch sizes exhibit the highest testing error rates of 0.383 and 0.380, respectively. These results are particularly notable as both methods showed rapid convergence in training. Faster convergence in SGD and AdaGrad could be leading to solutions that are optimal for the sampled batches but not for the overall data distribution, which may lead to poor generalization to the testing dataset.

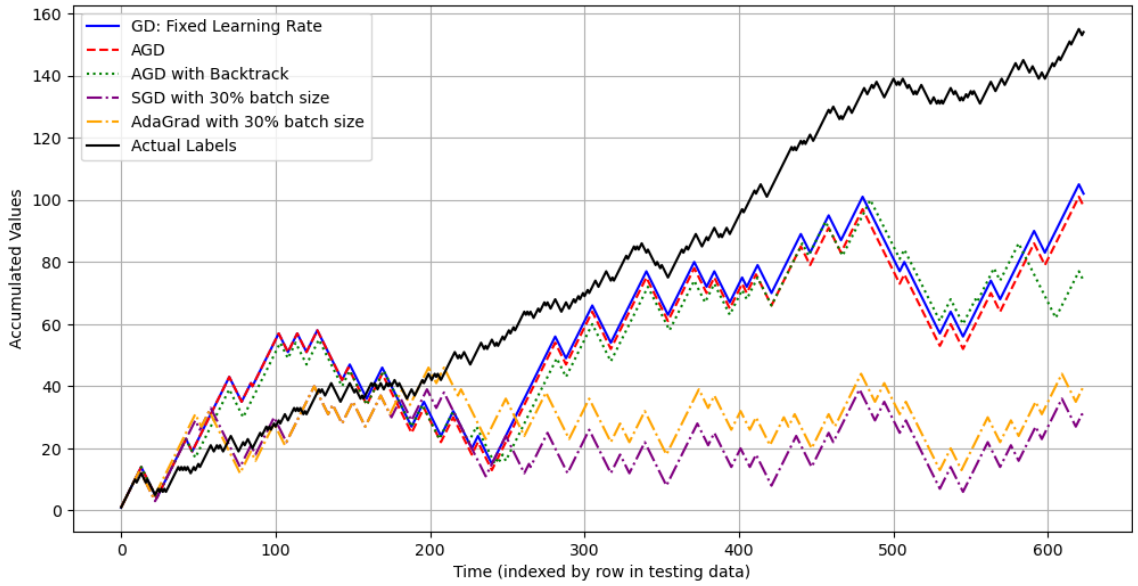


Figure 5: **Accumulative Testing Predictions:** The plot shows the accumulated predictions versus actual labels over time for various gradient descent methods by converting each correct prediction into +1 and each incorrect prediction into -1, then calculating the cumulative sum of these values

Another way to visualize the testing prediction is shown in Figure 5. This is plotted by converting each correct prediction into +1 and each incorrect prediction into -1, then calculating the cumulative sum of these values. This process is applied to the predictions from each gradient descent method as well as to the actual binary labels from the testing data. Although being slower in training, both GD and AGD track closer to the actual label curve than the other methods, suggesting a relatively better predictive consistency over the testing period. SGD with 30% batch size and AdaGrad with 30% batch size both significantly deviate from the actual labels, especially in the latter half of the series. This implies that both SGD and AdaGrad with a smaller batch size, although converge faster but at the cost of higher prediction volatility. While resampling smaller batches allow for quicker adjustments to weights, they might be over-fitting to subsets of the data or failing to generalize well across the entire data.

5 Conclusion

This study has analyzed the training efficiency and testing accuracy of various gradient descent methods in predicting weekly NASDAQ returns via logistic regression models. The

standard Gradient Descent (GD), with a fixed learning rate derived from the inverse of the Lipschitz constant, demonstrated the slowest convergence but maintained a steady reduction in loss and training error, and with the highest testing prediction accuracy. Accelerated Gradient Descent (AGD) greatly improved convergence speed without sacrificing two much on testing prediction accuracy, yet incorporating the backtracking line search added complexity that resulted in longer convergence times without significantly enhancing training efficiency.

On the other hand, Stochastic Gradient Descent (SGD) and AdaGrad exhibited divergent behaviors with varying batch sizes; particularly, smaller batch sizes led to substantially faster convergence but introduced higher volatility in prediction accuracy, which resulted in higher testing error rates. This indicate a potential trade-off of sophisticated GD algorithms, between training speed and the ability to generalize effectively to unseen data, on simple and straightforward classification problem. Based on the study's outcomes, it is suggested that for such simple classification tasks, AGD is considered as a relatively balanced approach for solving the optimization problem efficiently and precisely.

Bibliography

Armijo, L. (1966). Minimization of functions having Lipschitz continuous first partial derivatives. *Pacific Journal of Mathematics*, 16(1), 1–3. https://doi.org/10.2140/pjm.1966.16.1

Bianchi, D., Büchner, M., & Tamoni, A. (2019). What matters when? Time-Varying sparsity in expected returns. *Social Science Research Network*. https://doi.org/10.2139/ssrn.3438754

Christoffersen, P., & Diebold, F. X. (2006). Financial asset returns, Direction-of-Change forecasting, and volatility dynamics. *Management Science*, 52(8), 1273–1287. https://doi.org/10.1287/mnsc.1060.0520

Chung, J., & Hong, Y. S. (2007). Model-free evaluation of directional predictability in foreign exchange markets. *Journal of Applied Econometrics*, 22(5), 855–889. https://doi.org/10.1002/jae.965

Duchi, J. C., Hazan, E., & Singer, Y. (2011). Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. *Journal of Machine Learning Research (JMLR)*, 12(61), 2121–2159. http://digitalassets.lib.berkeley.edu/techreports/ucb/text/EECS-2010-24.pdf

Kendall, M. G., & Hill, A. (1953). The Analysis of Economic Time-Series-Part I: Prices. *Journal of the Royal Statistical Society. Series A, General*, 116(1), 11. https://doi.org/10.2307/2980947

Leung, M. T., Daouk, H., & Chen, A. S. (2000). Forecasting stock indices: a comparison of classification and level estimation models. *International Journal of Forecasting*, 16(2), 173–190. https://doi.org/10.1016/s0169-2070(99)00048-5

Linton, O., & Whang, Y. (2007). The quantilogram: With an application to evaluating directional predictability. *Journal of Econometrics*, 141(1), 250–282. https://doi.org/10.1016/j.jeconom.2007.01.004

Nesterov, Y. (1983). A method for solving the convex programming problem with convergence rate O(1/k$\hat{2}$). *Proceedings of the USSR Academy of Sciences*, 269, 543–547. https://ci.nii.ac.jp/naid/10029946121/

Nyberg, H. (2011). Forecasting the direction of the US stock market with dynamic binary probit models. *International Journal of Forecasting*, 27(2), 561–578. https://doi.org/10.1016/j.ijforecast.2010.02.008

Robbins, H., & Monro, S. (1951). A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3), 400–407. https://doi.org/10.1214/aoms/1177729586